



Sistemi di Calcolo Parallelo e Applicazioni

Progetto di fine corso: Prodotto Matrice Densa - Multivettore Ibrido

Simone Guidato

Matricola: 0354367

simone.guidato@students.uniroma2.eu

Anno Accademico 2025/2026

Indice

1	Introduzione e Specifiche del Problema	4
1.1	Ambiente di Test e Metriche	4
1.2	Strategia di Generazione Dati (Ripetibilità e Memoria Distribuita)	4
2	Distribuzione del Carico: Topologia e Comunicazione MPI	5
2.1	Topologia Cartesiana e Sotto-Comunicatori	5
2.2	Analisi del Pattern di Comunicazione	6
3	Ottimizzazione Intra-Nodo: OpenMP e CUDA	6
3.1	Elaborazione Multi-Core (OpenMP)	7
3.1.1	Kernel 0: Implementazione OpenMP Baseline	7
3.1.2	Kernel 1: Accumulo nei Registri e Vettorizzazione SIMD	8
3.2	Accelerazione Many-Core (CUDA)	9
3.2.1	Kernel 0: Implementazione CUDA Baseline	9
3.2.2	Kernel 1: Rimappatura degli Indici per Accessi Coalescenti	10
3.2.3	Kernel 2: Shared Memory Tiling e Read-Only Cache	11
3.2.4	Kernel 3: Warp-Row (Register Tiling e Warp-Broadcast)	12
3.2.5	Kernel 4: Trasposizione su GPU e Corner Turn per Scritture Coalescenti	13
4	Validazione e Benchmark di Riferimento	14
5	Analisi Sperimentale delle Prestazioni	15
5.1	Prestazioni su Architettura Host (CPU): OpenMP e Modello Ibrido	15
5.1.1	Picco Computazionale e Dominio dell'Approccio Ibrido	15
5.1.2	Analisi dello Speedup rispetto all'esecuzione Seriale	16
5.1.3	Scalabilità Vettoriale al variare di k e Impatto Geometrico	17
5.2	Prestazioni sull'Acceleratore (GPU): CUDA	19
5.2.1	Il ruolo di k e l'Intensità Aritmetica	19
5.2.2	Confronto tra i Kernel e Picco Computazionale	20
5.3	Comportamento Multi-Processo (MPI) e Oversubscription su GPU	22
5.3.1	Impatto della Topologia di Griglia $P \times Q$	23

5.4	Confronto Architetturale e Analisi End-to-End	23
6	Conclusioni	26

1 Introduzione e Specifiche del Problema

Il progetto in esame verte sulla realizzazione e ottimizzazione di un nucleo di calcolo HPC per il prodotto tra una matrice densa A e un multivettore X , per ottenere il multivettore risultato Y , secondo la relazione:

$$Y = A \cdot X$$

Un multivettore è definito come una matrice di dimensioni $N \times k$ (per X) e $M \times k$ (per Y), dove k assume valori piccoli. Nel rispetto delle specifiche progettuali, il codice è in grado di funzionare per dimensioni globali M ed N arbitrarie, scelte a *runtime* dall'utente, ed è stato collaudato estensivamente per valori di $k \in \{3, 6, 8, 20, 32, 64, 128\}$, coprendo così sia il set minimo richiesto dalla traccia che due valori aggiuntivi utili a osservare l'andamento delle prestazioni.

La matrice globale A viene distribuita su una griglia di processi cartesiana bidimensionale $P \times Q$; tale griglia può essere specificata esplicitamente dall'utente a runtime (parametri opzionali `grid.rows` e `grid.cols`), oppure calcolata automaticamente tramite `MPI_Dims_create` qualora non specificata. Questo consente, a parità di numero di processi totali, di confrontare topologie di griglia diverse (ad es. 1×8 , 8×1 , 2×4) e di analizzarne l'impatto sulle prestazioni.

Per l'analisi prestazionale sono stati considerati, oltre al caso quadrato ($M = N$, con $M \in \{2000, 4000\}$), quattro rapporti dimensionali aggiuntivi: $M = 2N$, $N = 2M$, $M = 3N$, $N = 3M$ (rispettivamente 4000×2000 , 2000×4000 , 6000×2000 , 2000×6000), per un totale di sei configurazioni distinte.

L'obiettivo principale è massimizzare le prestazioni computazionali attraverso una parallelizzazione a due livelli: un parallelismo distribuito tra processi tramite MPI e un parallelismo all'interno del singolo nodo tramite OpenMP (su CPU) e CUDA (su GPU).

1.1 Ambiente di Test e Metriche

Tutti i test prestazionali sono stati condotti sul server dipartimentale, caratterizzato dalla seguente architettura eterogenea:

- **CPU:** 2x Intel Xeon Silver 4210 (Architettura NUMA, 20 Core fisici, 40 Thread logici totali).
- **GPU:** 1x NVIDIA Quadro RTX 5000 (3072 CUDA Core, 16 GB VRAM GDDR6).

Come da specifiche, i tempi riportati non includono il preprocessamento dei dati, l'I/O su file o il trasferimento PCI da e verso la memoria del dispositivo CUDA. Le misurazioni sono state effettuate invocando ripetutamente il nucleo di calcolo e calcolando il tempo medio di esecuzione T . Il livello di prestazioni è quantificato in **GFLOPS**, utilizzando la formula:

$$\text{GFLOPS} = \frac{2 \cdot M \cdot N \cdot k}{T \cdot 10^9}$$

1.2 Strategia di Generazione Dati (Ripetibilità e Memoria Distribuita)

Per il collaudo funzionale e prestazionale, i dati sono stati generati *in-memory* avvalendosi di una strategia basata su formule matematiche deterministiche.

Al fine di garantire la totale **ripetibilità** dell'esperimento ed evitare colli di bottiglia legati all'allocazione dell'intera matrice A sul server a disposizione (che potrebbe causare errori di *Out Of Memory* per dimensioni elevate), si è optato per una **generazione puramente distribuita e stateless**.

Ogni processo MPI calcola i propri indici globali in base all'offset della sua posizione nella griglia cartesiana 2D e genera i valori dei propri elementi locali (A_{local} e X_{local}) applicando una formula dipendente dall'indice assoluto e da un *seed* globale fisso.

Listing 1: Generazione deterministica distribuita di A_{local}

```

1 long long global_idx=(info.offset_M + i) * info.global_N + (info.offset_N + j);
2 local_A[i * info.local_N + j]=(float)((global_idx + seed) % 100) / 7.0f;

```

Questo paradigma garantisce che una cella in coordinata globale (i, j) assuma l'esatto medesimo valore a prescindere dal numero di processi utilizzati per la computazione. In questo modo, la matrice A risulta nativamente e uniformemente distribuita sulla griglia di processi bidimensionale. Le uniche comunicazioni implementate sono relative al *Broadcast* e alla *Reduce* dei multivettori X e Y .

2 Distribuzione del Carico: Topologia e Comunicazione MPI

La parallelizzazione a livello distribuito è stata progettata per garantire scalabilità e minimizzare l'overhead di comunicazione, modellando il sistema su una griglia di processi bidimensionale cartesiana.

2.1 Topologia Cartesiana e Sotto-Comunicatori

Per mappare i processi MPI in modo coerente con la struttura delle matrici, è stata creata una topologia virtuale tramite `MPI_Cart_create`. Questa astrazione permette di gestire i processi non più come un array lineare di rank, ma come una matrice di coordinate (*row, col*). Fondamentale è stata la disattivazione del *reordering* per mantenere un controllo a livello più basso sul mapping tra i processi logici e le risorse fisiche del cluster.

La distribuzione dei dati è stata così partizionata:

- **Matrice A :** Suddivisa logicamente in blocchi A_{local} di dimensione $M_{loc} \times N_{loc}$, uno per ciascuno dei P processi della griglia. Tale suddivisione non comporta alcuna comunicazione di rete: ogni processo genera direttamente il proprio blocco a partire dagli offset di riga/colonna che gli competono nella griglia cartesiana.
- **Multivettore X :** Distribuito inizialmente solo lungo la **prima riga** della griglia ($rank_{row} = 0$), segmentato in porzioni da $N_{loc} \times k$.
- **Multivettore Y :** Risultato finale consolidato esclusivamente sulla **prima colonna** della griglia ($rank_{col} = 0$).

L'orchestrazione dei dati richiede scambi specifici per righe e per colonne. Per evitare contese sul bus di rete e ottimizzare il throughput, la griglia è stata divisa in **sotto-comunicatori**

ortogonali (*Row Communicators* e *Column Communicators*) tramite la funzione `MPI_Cart_sub`. Questo approccio permette di limitare la moltitudine delle operazioni collettive solo ai processi strettamente necessari.

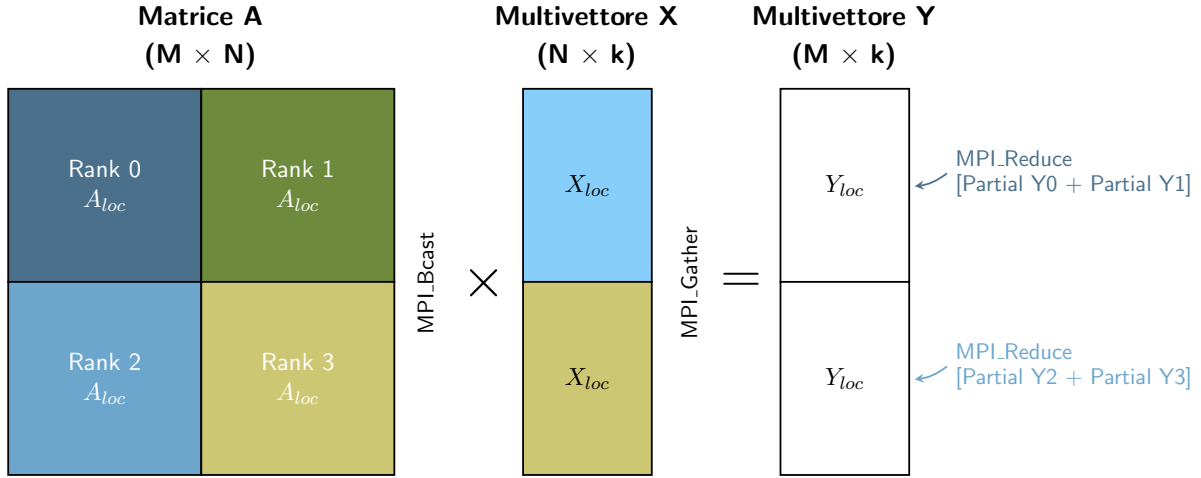


Figura 1: Schema a blocchi del prodotto Matrice-Multivettore su griglia MPI 2x2.

2.2 Analisi del Pattern di Comunicazione

L'algoritmo di calcolo distribuito è stato strutturato in tre fasi distinte e sequenziali:

1. **Fase 1: Generazione e Broadcast del Multivettore X.** Analogamente alla matrice A , ciascun processo con $rank_{row} = 0$ genera direttamente in locale la propria porzione di X (stessa formula deterministica e stateless usata per A), senza alcuna comunicazione di rete. Successivamente, ogni testa delle colonne propaga il proprio frammento agli altri processi della colonna tramite un'unica `MPI_Bcast` lungo il *Column Sub-Communicator*. L'unica comunicazione MPI in questa fase è dunque la Bcast, non uno scatter con `MPI_Send` punto-a-punto: questa scelta evita completamente il costo di serializzazione/invio dal Rank 0, limitando il traffico di rete al minimo indispensabile.
2. **Fase 2: Computazione Locale.** Una volta ottenuti i dati, ogni processo esegue il kernel di calcolo $Y_{parziale} = A_{loc} \times X_{loc}$. In questa fase non avviene alcuna comunicazione MPI, sfruttando appieno il parallelismo intra-nodo (OpenMP/CUDA) e garantendo che il calcolo sia perfettamente bilanciato.
3. **Fase 3: Riduzione Orizzontale (Row-wise Reduce).** Il multivettore Y finale è la somma dei contributi parziali calcolati dai processi sulla stessa riga. Viene quindi eseguita una `MPI_Reduce` attraverso una `MPI_SUM` lungo i sotto-comunicatori di riga. Il destinatario della riduzione è il processo "testa" di riga (sulla prima colonna). Infine, questi processi raccolgono e inviano le strisce risultanti al Rank 0 globale tramite una `MPI_Gatherv` lungo il comunicatore di colonna.

3 Ottimizzazione Intra-Nodo: OpenMP e CUDA

L'efficacia di un'applicazione distribuita dipende anche da quanto velocemente ogni singolo nodo elabora i propri dati. Poiché l'hardware si comporta in modo diverso a seconda dell'architettura,

il calcolo è stato implementato in due modi: su CPU multi-core tramite OpenMP e su GPU tramite CUDA.

3.1 Elaborazione Multi-Core (OpenMP)

Su CPU, il problema principale non è il numero di operazioni, ma la velocità con cui i dati viaggiano dalla memoria ai registri (limite *memory-bound*).

Indipendentemente dalle specifiche ottimizzazioni applicate ai cicli, il flusso di esecuzione base dell'implementazione OpenMP è il seguente:

1. **Acquisizione dati:** Il processo MPI riceve la propria sottomatrice A_{loc} e la porzione del multivettore X_{loc} in RAM.
2. **Fork:** Il thread master apre una regione parallela, generando un team di thread a cui viene spartito il lavoro (di solito le righe della matrice).
3. **Computazione:** I thread calcolano il prodotto e salvano i risultati parziali in Y_{loc} , cercando di massimizzare i *cache hit*.
4. **Join e Riduzione:** I thread si sincronizzano a fine calcolo. Il processo MPI master riprende il controllo e partecipa alla `MPI_Reduce` collettiva per unire i risultati.

3.1.1 Kernel 0: Implementazione OpenMP Baseline

La prima implementazione su CPU costituisce la versione di riferimento dello studio prestazionale. L'algoritmo mantiene la classica struttura tripla del prodotto matrice-multivettore e introduce unicamente una parallelizzazione a livello delle righe della matrice mediante OpenMP.

La direttiva

```
1 #pragma omp parallel for schedule(static)
```

distribuisce uniformemente le righe della matrice tra i thread disponibili. Ogni thread è quindi responsabile del calcolo di un insieme di righe complete della matrice risultato Y , evitando qualsiasi forma di sincronizzazione durante la fase computazionale. Poiché ciascun thread scrive esclusivamente nelle righe di propria competenza, non si verificano conflitti di scrittura (*race conditions*) e non sono necessarie operazioni di riduzione.

Per ogni riga assegnata, il kernel inizializza preventivamente gli elementi corrispondenti del multivettore risultato a zero. Successivamente viene eseguito il classico triplo ciclo annidato

$$Y_{i,p} = \sum_{j=0}^{N-1} A_{i,j} \cdot X_{j,p},$$

dove l'elemento della matrice $A(i, j)$ viene caricato una sola volta nella variabile locale `a_val` e riutilizzato per il calcolo di tutte le k colonne del multivettore in modo tale da ridurre il numero di accessi alla memoria principale.

L'utilizzo della politica di scheduling statico (`schedule(static)`) garantisce una ripartizione uniforme del lavoro tra i thread, particolarmente adatta al problema in esame, poiché ogni riga richiede lo stesso numero di operazioni.

Dal punto di vista della gerarchia di memoria, il kernel sfrutta la località spaziale durante la scansione sequenziale delle righe della matrice A e delle colonne del multivettore X . Tuttavia, non vengono ancora applicate tecniche migliorative o altre ottimizzazioni orientate al riutilizzo dei dati.

3.1.2 Kernel 1: Accumulo nei Registri e Vettorizzazione SIMD

La seconda implementazione introduce una serie di ottimizzazioni mirate a migliorare l'efficienza dell'esecuzione sulle moderne CPU multi-core, mantenendo invariata la distribuzione del lavoro tra i thread OpenMP. Anche in questo caso, il parallelismo viene ottenuto assegnando ai thread blocchi distinti di righe della matrice mediante

```
1 #pragma omp parallel for schedule(static)
```

poiché il carico computazionale associato a ciascuna riga è uniforme.

La principale differenza rispetto alla versione baseline consiste nell'introduzione di un buffer locale

```
1 float y_local[MAX_K] = {0};
```

allocato sullo stack di ciascun thread. Poiché la dimensione massima del multivettore è nota a tempo di compilazione ($MAX_K = 128$), il compilatore può mantenere gran parte del vettore risultato nei registri della CPU durante l'intero prodotto scalare.

Per ogni elemento della matrice, il valore

```
1 float a_val = A[i * N + j];
```

viene letto una sola volta dalla memoria e successivamente riutilizzato per aggiornare tutte le componenti del multivettore locale. Al termine dell'elaborazione della riga, il contenuto di `y_local` viene copiato nella matrice risultato mediante un'unica scansione sequenziale.

Per favorire ulteriormente le ottimizzazioni del compilatore, i cicli interni sono accompagnati da

```
1 #pragma omp simd safelen(MAX_K)
```

che informa il compilatore dell'assenza di dipendenze tra le iterazioni e consente la generazione di istruzioni SIMD (Single Instruction Multiple Data). Sulle architetture x86 moderne, tale direttiva permette di sfruttare le estensioni vettoriali AVX2 o AVX-512, elaborando simultaneamente più elementi del multivettore mediante un'unica istruzione.

L'utilizzo combinato dei registri e della vettorizzazione riduce significativamente il traffico verso la memoria principale. Ogni elemento della matrice A viene caricato una sola volta e riutilizzato per l'aggiornamento di tutte le componenti del vettore locale, mentre la scrittura sulla matrice risultato avviene esclusivamente al termine del calcolo della riga. Rispetto alla versione baseline, diminuisce quindi il numero complessivo di accessi alla memoria e aumenta il parallelismo a livello di istruzioni (*Instruction Level Parallelism*), consentendo alla CPU di sfruttare in maniera più efficace le proprie unità vettoriali.

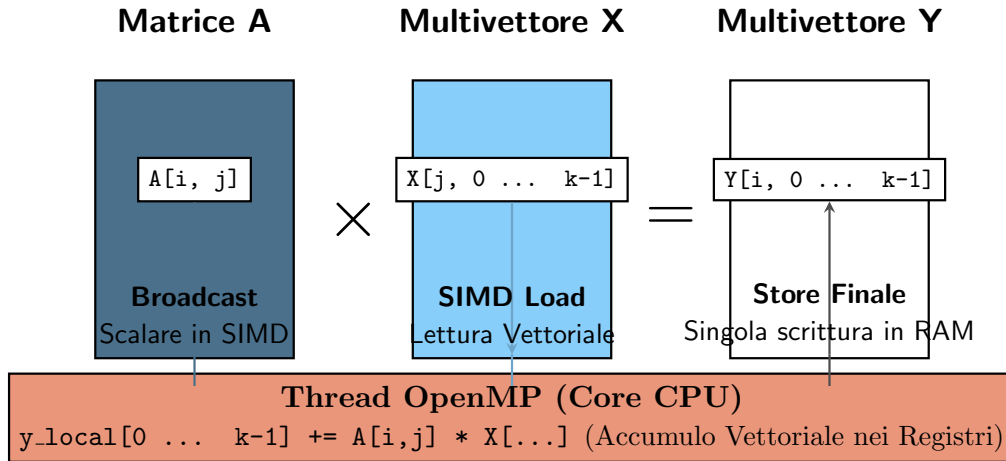


Figura 2: Schema del pattern di accesso in memoria per l'approccio OpenMP SIMD. Il Thread CPU preleva lo scalare da A in broadcast, vettorizza la riga di X e accumula interamente i risultati nell'array locale `y_local` situato nei registri, abbattendo le transazioni ripetute verso la memoria RAM.

3.2 Accelerazione Many-Core (CUDA)

Sulla GPU, l'alta latenza della memoria viene nascosta eseguendo contemporaneamente migliaia di thread raggruppati in blocchi e Warp.

L'uso della GPU cambia il flusso di calcolo del nodo, perché richiede di gestire manualmente i trasferimenti di dati sul bus PCI-Express e di definire la griglia di esecuzione:

1. **Allocazione e Trasferimento (H2D):** Una volta ricevuti i dati via MPI, il processo alloca spazio nella *Global Memory* della GPU e copia i buffer di A_{loc} e X_{loc} (Host-to-Device).
2. **Offloading:** Il processore host configura la griglia di calcolo (**Grid** e **Block dimension**) e lancia il kernel specifico in modo asincrono.
3. **Esecuzione:** La GPU esegue il prodotto matrice-multivettore. In questa fase entra in gioco lo sfruttamento della memoria gerarchica on-chip (*Shared Memory* e registri) per ottimizzare le letture.
4. **Recupero dati:** Al termine dell'esecuzione, il vettore risultante Y_{loc} viene copiato dalla VRAM alla RAM di sistema per essere infine affidato alla rete MPI.

3.2.1 Kernel 0: Implementazione CUDA Baseline

Il Kernel 0 rappresenta la prima implementazione del prodotto matrice-multivettore sulla GPU. La strategia adottata è quella classica *one thread-one output*: ogni thread CUDA è responsabile del calcolo di un singolo elemento della matrice risultato Y , identificato dalla coppia di coordinate (*row, col*).

La configurazione di lancio utilizza una griglia bidimensionale di blocchi composta da blocchi di dimensione 32×8 thread:

```
1 dim3 threadsPerBlock(32, 8);
```

```

2
3 int col = blockIdx.x * blockDim.x + threadIdx.x;
4 int row = blockIdx.y * blockDim.y + threadIdx.y;

```

Per ogni elemento assegnato, il thread esegue il prodotto scalare tra la riga corrispondente della matrice A e la colonna del multivettore X :

$$Y_{row,col} = \sum_{j=0}^{N-1} A_{row,j} \cdot X_{j,col}.$$

Il risultato parziale viene accumulato in una variabile locale (`sum`), che il compilatore alloca nei registri del singolo thread. Al termine del ciclo, viene effettuata un'unica scrittura nella memoria globale:

```

1 float sum = 0.0f;
2
3 for (int j = 0; j < N; j++)
4     sum += A[row * N + j] * X[j * k + col];
5
6 Y[row * k + col] = sum;

```

L'organizzazione dei thread rende inoltre favorevoli gli accessi alla memoria globale. I thread appartenenti allo stesso warp condividono lo stesso indice di riga e presentano valori consecutivi dell'indice di colonna; di conseguenza, le letture del multivettore X e le scritture sulla matrice risultato Y risultano naturalmente coalescenti, mentre l'accesso alla matrice A viene condiviso tra tutti i thread del warp, favorendo il riutilizzo dei dati nella cache della GPU.

Nonostante tali caratteristiche, il kernel continua a effettuare tutte le operazioni direttamente sulla memoria globale. Ogni elemento della matrice A e del multivettore X viene ricaricato a ogni iterazione del prodotto scalare, limitando l'intensità aritmetica e rendendo il kernel prevalentemente *memory-bound*. Questa implementazione costituisce quindi il riferimento base rispetto alle versioni ottimizzate sviluppate successivamente.

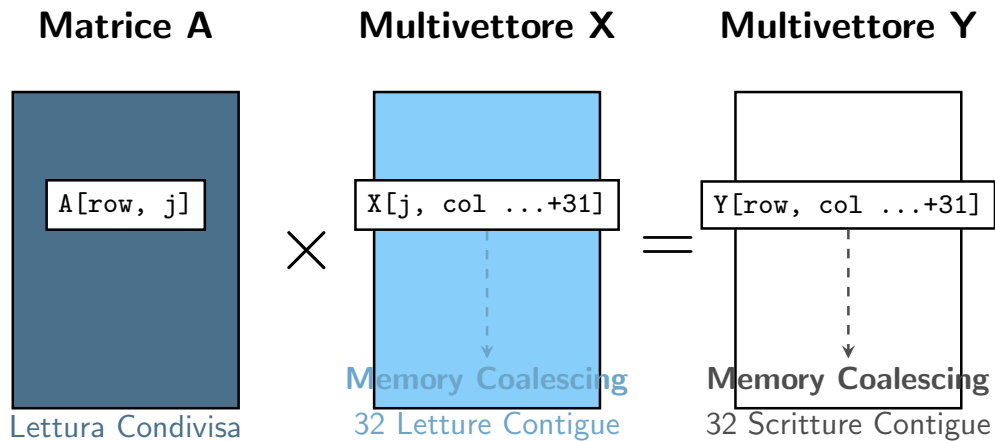


Figura 3: Pattern di accesso in memoria per un singolo Warp.

3.2.2 Kernel 1: Rimappatura degli Indici per Accessi Coalescenti

Il Kernel 1 mantiene l'approccio *un thread per elemento*. L'obiettivo qui non è cambiare l'algoritmo, ma osservare come la GPU gestisce i thread e gli accessi in memoria.

Come visto nel Kernel 0, il mapping a 2D fa sì che i thread dello stesso warp leggano la stessa riga di A e colonne contigue di X . Di conseguenza, le letture su X e le scritture su Y sono naturalmente coalescenti, mentre la lettura di A sfrutta il broadcast della cache L1. Questo dimostra che basta organizzare bene i thread per sfruttare la banda di memoria disponibile.

Tuttavia, leggendo sempre direttamente dalla memoria globale senza usare la Shared Memory, le prestazioni di questo kernel restano limitate dalla latenza.

3.2.3 Kernel 2: Shared Memory Tiling e Read-Only Cache

Il Kernel 2 rappresenta la principale ottimizzazione dell'implementazione CUDA. A differenza dei kernel precedenti, che effettuano tutti gli accessi direttamente alla memoria globale, questa versione introduce una strategia di *tiling* basata sull'utilizzo della memoria condivisa (*Shared Memory*) e della cache in sola lettura della GPU.

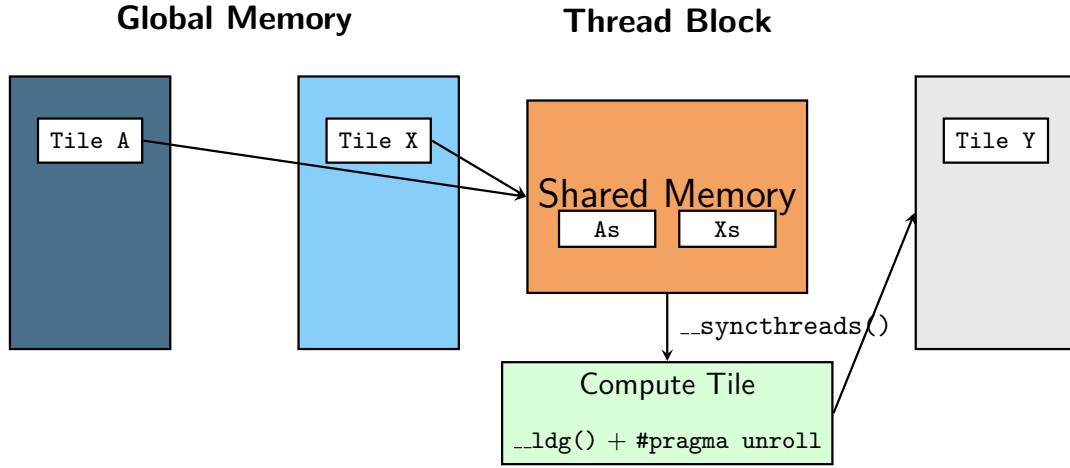
L'idea fondamentale consiste nel suddividere la moltiplicazione matrice-multivettore in una sequenza di sotto-problemi di dimensione fissa (`TILE_SIZE = 32`). Ogni blocco di thread coopera nel trasferimento di una porzione della matrice A e del multivettore X dalla memoria globale alla memoria condivisa:

```
1 __shared__ float As[TILE_SIZE][TILE_SIZE];  
2 __shared__ float Xs[TILE_SIZE][TILE_SIZE];
```

Una volta completato il caricamento, sincronizzato mediante `__syncthreads()`, ciascun thread esegue i prodotti scalari utilizzando esclusivamente i dati residenti nella Shared Memory, riducendo drasticamente il numero di accessi alla memoria globale.

Per migliorare ulteriormente l'efficienza delle letture, gli accessi agli operandi vengono effettuati tramite l'istruzione intrinseca `__ldg()`, che permette di sfruttare la cache in sola lettura dell'architettura CUDA. Questa tecnica riduce ulteriormente la latenza degli accessi, in particolare quando gli stessi dati vengono riutilizzati da più thread appartenenti al medesimo blocco.

L'iterazione interna sul contenuto della tile viene inoltre annotata mediante la direttiva `#pragma unroll`, consentendo al compilatore di espandere completamente il ciclo e ridurre il numero di istruzioni di controllo, aumentando così il parallelismo a livello di istruzioni (*Instruction Level Parallelism*).



Ogni tile viene caricata una sola volta dalla memoria globale, riutilizzata da tutti i thread del blocco e infine scritta su Y .

Figura 4: Schema del Kernel 2. Le tile della matrice A e del multivettore X vengono trasferite dalla memoria globale alla Shared Memory.

Nel complesso, questa implementazione aumenta l'intensità aritmetica del kernel: ogni elemento trasferito dalla memoria globale viene riutilizzato più volte dai thread del blocco prima di essere scartato, diminuendo il rapporto tra traffico di memoria e operazioni floating-point.

3.2.4 Kernel 3: Warp-Row (Register Tiling e Warp-Broadcast)

Il Kernel 3 cambia strategia: l'unità di calcolo di base diventa l'intero warp. Invece di mappare un thread per ogni elemento, un intero warp calcola una riga di A , e ogni suo thread si occupa di un elemento diverso della riga risultato in Y .

Il mapping si ottiene fissando `blockDim = (32, 8)`. Il valore `warp_id` identifica la riga della matrice assegnata al warp, mentre `lane_id` individua il singolo thread all'interno del warp.

Poiché il warp lavora su una sola riga, la cache fa il broadcast del singolo elemento di A a tutti i 32 thread in contemporanea (come spiegato nel Kernel 0). In questa variante, però, non usiamo la Shared Memory. Evitiamo così il costo di sincronizzazione (`__syncthreads()`) e accumuliamo il prodotto scalare direttamente nei registri locali del thread (`sum`). Le letture su X restano perfettamente coalescenti.

Questa strategia è molto efficace per ridurre i trasferimenti in memoria globale, specialmente quando il numero di colonne del multivettore è piccolo abbastanza da essere gestito da un singolo warp.

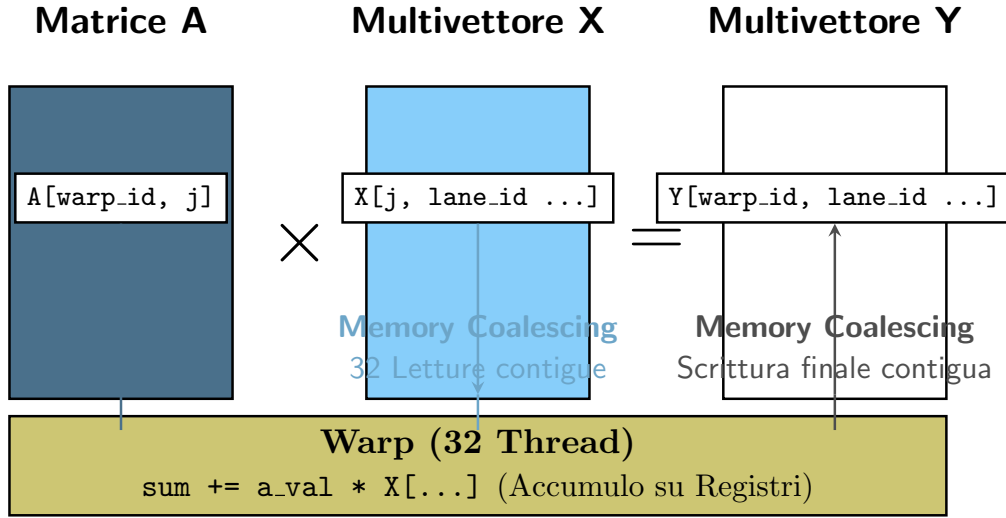


Figura 5: Schema del pattern di accesso in memoria per l'approccio Warp-Row.

3.2.5 Kernel 4: Trasposizione su GPU e Corner Turn per Scritture Coalescenti

Il Kernel 4 affronta un problema rimasto irrisolto nei Kernel 1 e 3: il mapping che rende coalescenti le letture (riga come dimensione del warp) rende invece le scritture su Y non contigue, perché in memoria Y è organizzata come $Y[\text{row} \cdot k + \text{col}]$ e thread consecutivi, con row diverso e stesso col , scrivono a indirizzi distanti k elementi.

Per risolverlo, il kernel lavora in due fasi.

Prima trasponiamo A direttamente sulla GPU con un kernel dedicato, invece che sull'host. Il kernel carica un tile 32×32 in Shared Memory e lo riscrive trasposto, con un padding di +1 sulla tile per evitare i bank conflicts:

```

1 __shared__ float tile[TILE_DIM][TILE_DIM + 1];
2 tile[threadIdx.y][threadIdx.x] = idata[y * N + x];
3 // ... dopo __syncthreads(), riscrittura trasposta in odata

```

Poi, nel calcolo vero e proprio, ogni thread legge da A^T con lettura coalescente, esattamente come nel Kernel 3, e accumula il prodotto scalare in un registro. Successivamente, invece di scrivere subito su Y , il risultato viene depositato in una tile di Shared Memory:

```

1 smem_Y[threadIdx.y][threadIdx.x] = sum;
2 __syncthreads();

```

A questo punto i thread si scambiano i ruoli: chi prima aveva la riga come indice veloce ora ha la colonna, e viceversa. Si rilegge la Shared Memory con gli indici invertiti e si scrive il risultato finale su Y , che risulta così perfettamente coalescente anche in scrittura:

```

1 int out_col_k = blockIdx.y * TILE_DIM + threadIdx.x;
2 d_Y[out_row_i * k + out_col_k] = smem_Y[threadIdx.x][threadIdx.y];

```

Rispetto al Kernel 3, il Kernel 4 elimina quindi anche l'ultima asimmetria rimasta tra lettura e scrittura. Resta comunque, come i precedenti, un kernel un thread-un elemento: non introduce tiling sul ciclo di riduzione, quindi non aumenta l'intensità aritmetica rispetto a Tiled (Kernel 2). Il costo è una sincronizzazione in più e una piccola porzione di Shared Memory usata solo per rimescolare l'output, non per il riutilizzo di A o X .

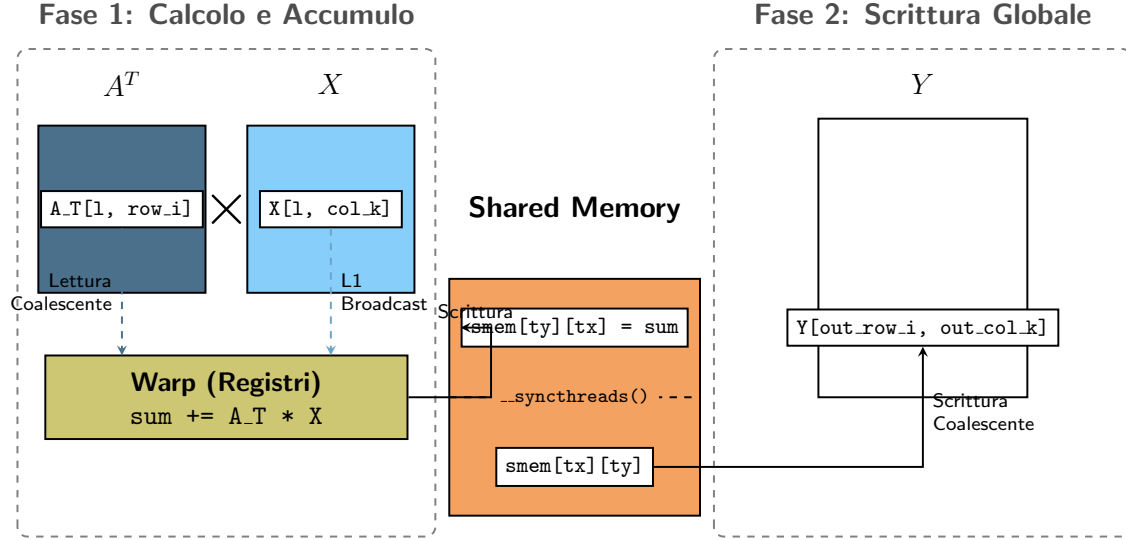


Figura 6: Schema del Kernel 4. Fase 1: lettura coalescente da A^T e accumulo nei registri. Fase 2: passaggio in Shared Memory per rimescolare le coordinate hardware (*Corner Turn*) seguito dalla trascrizione contigua e coalescente in memoria globale su Y .

4 Validazione e Benchmark di Riferimento

Per garantire la correttezza delle implementazioni parallele sviluppate (MPI, OpenMP e CUDA), è stata realizzata una versione seriale di riferimento utilizzata esclusivamente durante la fase di validazione. Tale implementazione costituisce il modello matematico rispetto al quale vengono confrontati tutti i risultati prodotti dalle versioni parallele.

La procedura di validazione viene eseguita soltanto quando l'opzione `do_validate` è abilitata. In tal caso, il processo master ricostruisce integralmente la matrice globale A e il multivettore X utilizzando le stesse formule deterministiche impiegate durante l'esecuzione distribuita. Questa scelta garantisce che entrambe le implementazioni operino esattamente sugli stessi dati, senza richiedere alcun caricamento da file o operazioni di comunicazione aggiuntive.

Successivamente viene eseguito il prodotto matrice-multivettore mediante l'algoritmo seriale, misurandone il tempo di esecuzione tramite la funzione `MPI_Wtime()`. Il risultato ottenuto viene quindi confrontato con quello prodotto dall'implementazione parallela attraverso il calcolo dell'errore relativo

$$\varepsilon = \frac{\|Y_{serial} - Y_{parallel}\|_2}{\|Y_{serial}\|_2},$$

L'implementazione viene considerata corretta quando l'errore relativo risulta inferiore alla soglia

$$\varepsilon < 10^{-5},$$

compatibile con la precisione in virgola mobile a singola precisione (`float`). Oltre alla verifica numerica, l'esecuzione seriale costituisce anche il riferimento utilizzato per il calcolo dello *speedup*. Indicando con T_{serial} il tempo dell'implementazione sequenziale e con $T_{parallel}$ il

tempo medio dell'implementazione parallela, lo speedup viene calcolato come

$$S = \frac{T_{serial}}{T_{parallel}},$$

fornendo una misura diretta del beneficio introdotto dalle strategie di parallelizzazione sviluppate durante il progetto. Su tutte le configurazioni della campagna sperimentale (oltre 4000 combinazioni di kernel, dimensione, k e numero di processi), l'errore relativo massimo osservato è pari a 6.8×10^{-6} , ben al di sotto della soglia $\varepsilon < 10^{-5}$: un margine di quasi un ordine di grandezza che conferma la robustezza numerica dell'implementazione indipendentemente dalla configurazione impiegata.

5 Analisi Sperimentale delle Prestazioni

Tutti i test sono stati misurati in GFLOPS, isolando il tempo di calcolo netto dai costi di inizializzazione dell'ambiente.

Per restituire una visione completa, l'indagine esamina prima l'elaborazione multi-core su CPU (OpenMP e MPI), successivamente l'accelerazione many-core su GPU (CUDA), per poi convergere in uno scontro diretto architetturale arricchito dall'analisi dei tempi end-to-end.

5.1 Prestazioni su Architettura Host (CPU): OpenMP e Modello Ibrido

La CPU fa affidamento su un numero ristretto di core fisici (20 totali nel nostro ambiente di test) dotati di frequenze più elevate e unità vettoriali avanzate (SIMD). I test mettono a confronto l'esecuzione puramente seriale (**Serial**), l'approccio a memoria condivisa (**SMP** - Symmetric Multiprocessing con soli thread OpenMP), l'approccio puramente distribuito (**Grid** - con soli processi MPI) e infine l'approccio **Hybrid** (MPI + OpenMP).

5.1.1 Picco Computazionale e Dominio dell'Approccio Ibrido

L'efficacia delle ottimizzazioni a livello di registro emerge chiaramente osservando i picchi massimi di *throughput*.

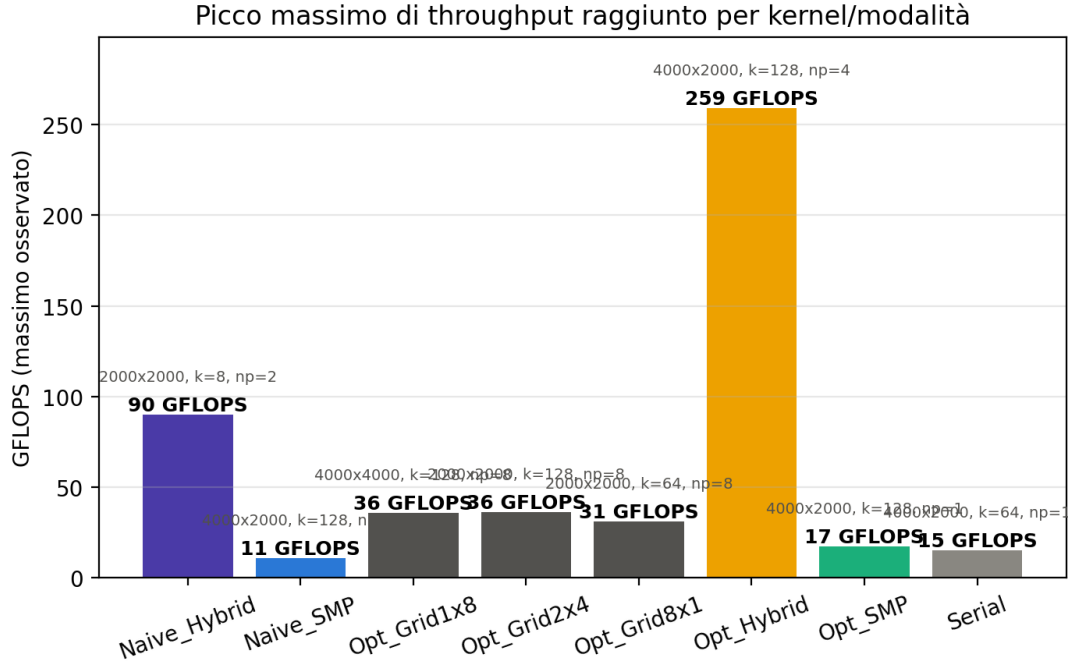


Figura 7: Picco massimo assoluto di throughput (GFLOPS) registrato per ciascun approccio su CPU.

Come illustrato in Figura 7, l'implementazione seriale di base si ferma a un modesto tetto di 15 GFLOPS. L'applicazione della sola vettorizzazione (`Opt_SMP`) migliora le prestazioni, ma è l'integrazione del modello **Hybrid** (**MPI + OpenMP**) a far registrare un notevole salto prestazionale, raggiungendo un picco di **259 GFLOPS** sulla configurazione 4000×2000 con $k = 128$. Questo dominio assoluto dell'ibrido è giustificato dall'architettura *NUMA* del server: utilizzando più processi MPI (es. $np = 4$), il sistema operativo è in grado di distribuire i processi e i relativi thread OpenMP sui diversi socket fisici, allocando la memoria localmente al processore che la sta elaborando e massimizzando la banda di memoria RAM disponibile.

5.1.2 Analisi dello Speedup rispetto all'esecuzione Seriale

Per quantificare il guadagno reale della parallelizzazione, è stato calcolato lo *Speedup* normalizzato rispetto al tempo di esecuzione dell'algoritmo seriale.

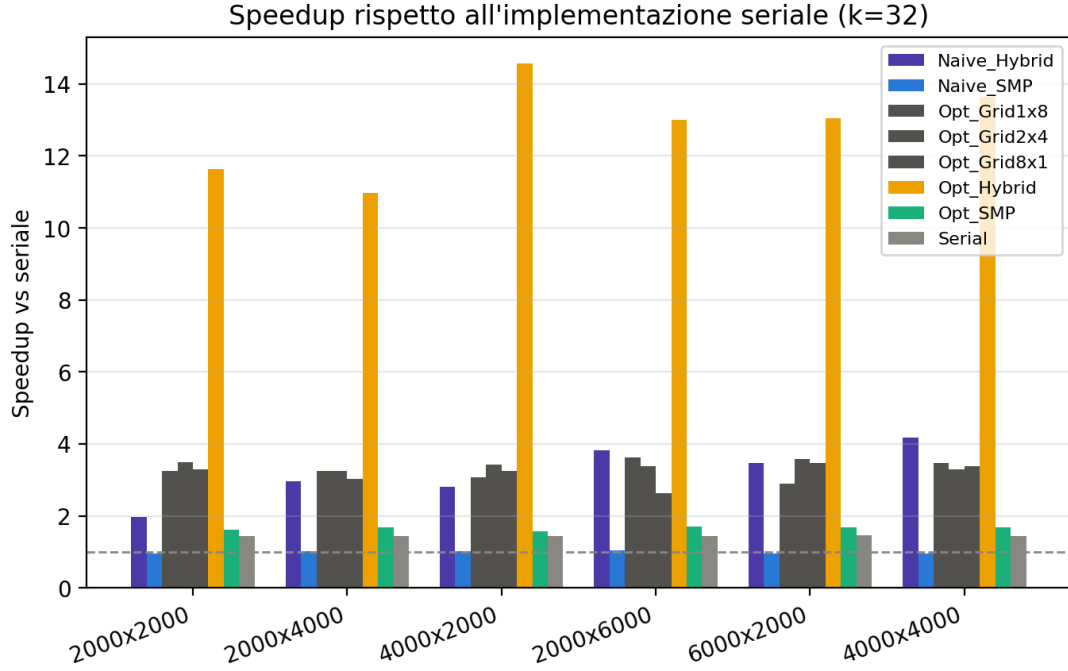


Figura 8: Speedup delle varie configurazioni rispetto all'implementazione seriale a $k = 32$.

Il grafico in Figura 8 evidenzia che la configurazione `Opt_Hybrid` riesce a mantenere uno *Speedup* compreso tra **11x e 14.5x** in quasi tutte le configurazioni di test, sebbene non si raggiunga uno scaling idealmente lineare (che corrisponderebbe a 20x per 20 core fisici) a causa della saturazione del bus di memoria principale. Le varianti `Opt_Grid` (puramente MPI) mostrano speedup limitati, confermando che frammentare eccessivamente i dati tramite IPC sulla stessa macchina genera un overhead che vanifica il calcolo.

5.1.3 Scalabilità Vettoriale al variare di k e Impatto Geometrico

Il parametro k gioca un ruolo cruciale sul versante CPU, dove funge da "moltiplicatore" per l'efficacia delle unità vettoriali (SIMD).



Figura 9: Andamento del throughput su CPU al variare del numero di colonne k (2000×2000).

La Figura 9 dimostra come l'approccio ibrido ottimizzi l'incremento di k . Passando da $k = 3$ a $k = 64$, il throughput cresce quasi linearmente. Per valori di k troppo piccoli, i registri vettoriali rimangono parzialmente vuoti, sprecando cicli di clock. All'aumentare di k , il compilatore riesce a riempire completamente le pipeline della ALU, massimizzando il numero di operazioni eseguite per singola istruzione. A differenza della singola configurazione 2000×2000 mostrata in precedenza, la vista completa sulle sei dimensioni conferma che il comportamento di **Opt.Hybrid** rispetto a k è coerente su tutte le geometrie testate: la crescita quasi lineare tra $k = 3$ e $k = 64$ non è un effetto isolato di una singola taglia di matrice, ma un pattern strutturale legato alla saturazione delle unità SIMD, indipendente dalla forma Tall/Square/Wide della matrice.

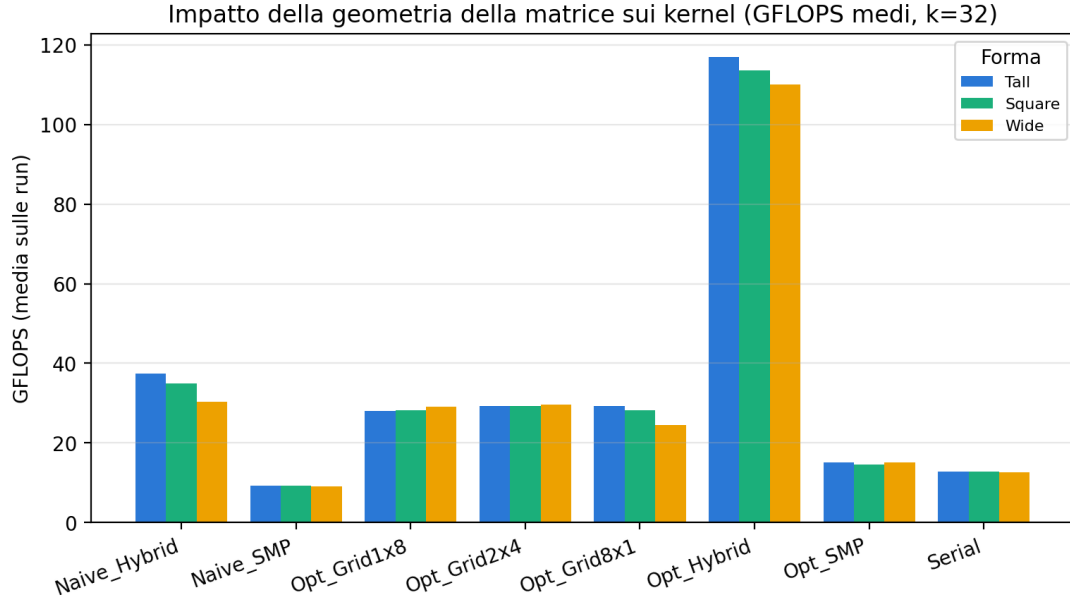


Figura 10: Effetto della forma della matrice sul throughput CPU ($k = 32$, a parità di elementi totali).

Inoltre, il grafico in Figura 10 sancisce una chiara preferenza per le geometrie **Tall** ($M > N$) e **Square** ($M = N$) a discapito di quelle **Wide**. Questo deriva dalla direttiva di scheduling (`schedule(static)`) applicata sulle righe: un numero di righe M maggiore permette allo *scheduler* OpenMP di suddividere il carico di lavoro in *chunk* di memoria più ampi, riducendo l'overhead di attivazione dei thread e minimizzando il *False Sharing* nelle cache.

5.2 Prestazioni sull'Acceleratore (GPU): CUDA

Spostando l'analisi sull'architettura della NVIDIA Quadro RTX 5000, cambiano i paradigmi e i tetti prestazionali. In questo scenario, le logiche di mascheramento della latenza e di coalescenza della memoria assumono un ruolo predominante.

5.2.1 Il ruolo di k e l'Intensità Aritmetica

Il grafico in Figura 11 mostra l'andamento del throughput di tutti i kernel CUDA all'aumentare di k , su tutte e sei le configurazioni dimensionali testate.

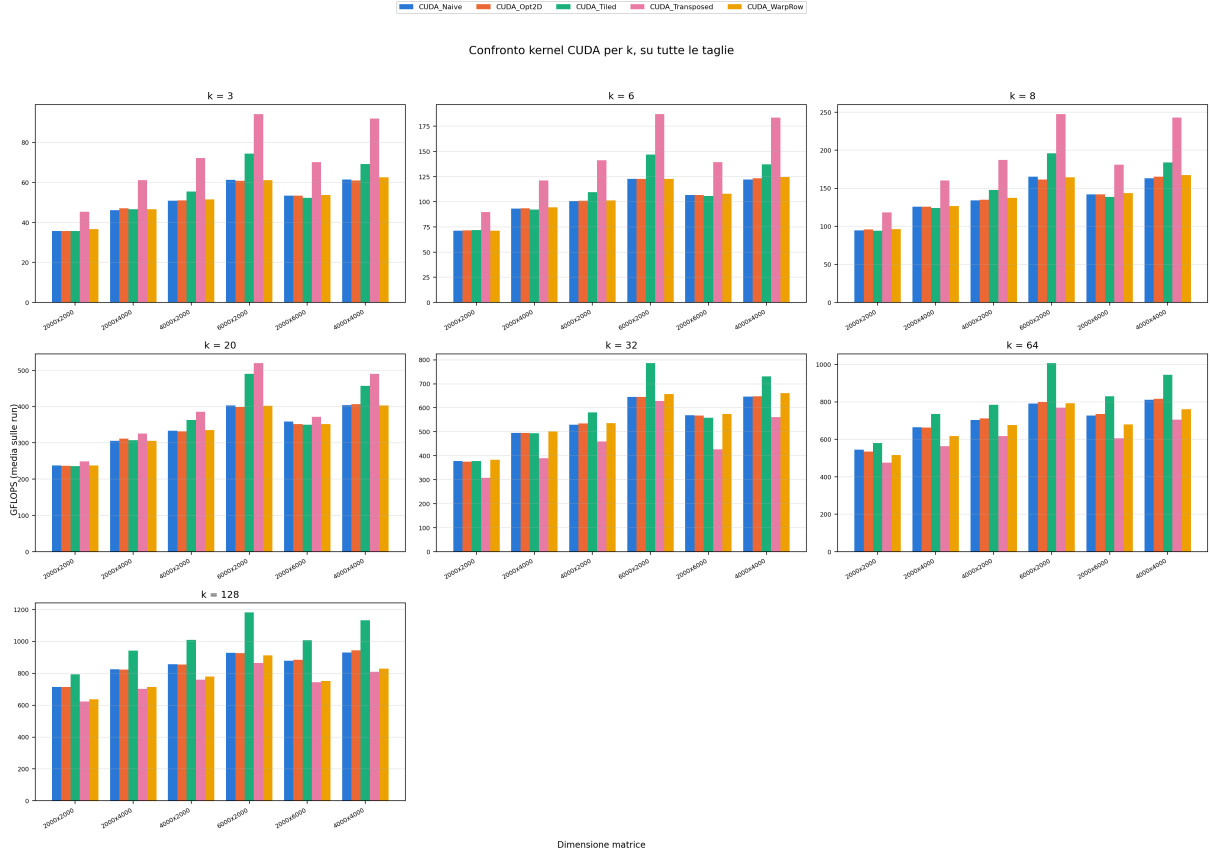


Figura 11: Confronto del throughput tra i vari kernel CUDA al variare delle colonne del multi-vettore (k), su tutte le dimensioni testate.

È evidente un andamento di crescita rapida per tutti i kernel ottimizzati nel range $k \in [3, 32]$, che tende poi ad assestarsi. Per valori di k piccoli, il tempo di esecuzione è limitato dalle latenze di trasferimento dalla VRAM (*Memory-Bound*). All'aumentare di k , la mole di calcoli per singolo dato caricato cresce, permettendo agli *Scheduler Warp* della GPU di nascondere le latenze eseguendo operazioni matematiche su altri warp attivi (*Latency Hiding*).

La vista aggregata su tutte le dimensioni rivela inoltre un dettaglio che la sola matrice 2000×2000 non mostrava da solo: per $k \in \{3, 6, 8\}$ il kernel **Transposed** è sistematicamente il migliore su tutte e sei le configurazioni, non un caso isolato di una singola geometria — il vantaggio dell'accesso completamente coalescente domina quando il lavoro per thread è ridotto. Il sorpasso da parte di **Tiled** avviene invece in modo consistente a partire da $k \approx 20-32$ su ogni taglia testata: il crossover tra le due strategie è quindi governato dal valore di k , non dalla forma della matrice.

5.2.2 Confronto tra i Kernel e Picco Computazionale

L'evoluzione delle ottimizzazioni *hardware-aware* studiate nell'implementazione si riflette nettamente nei valori di picco.

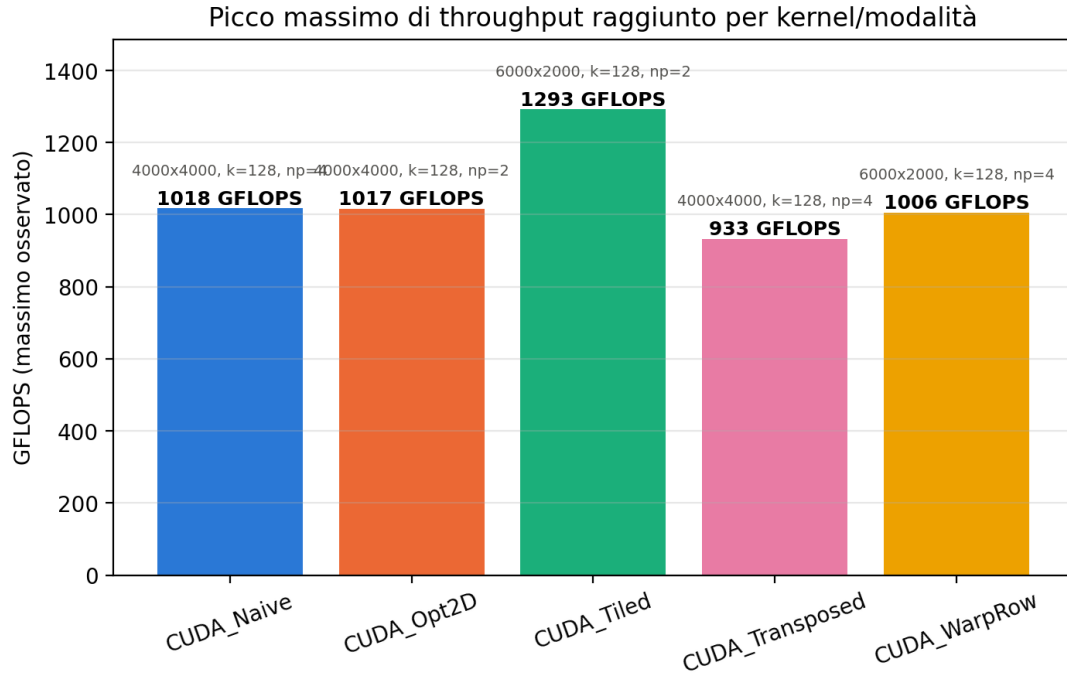


Figura 12: Picco massimo assoluto di throughput (GFLOPS) registrato per ciascun kernel CUDA.

Come illustrato in Figura 12, il **Kernel Tiled** emerge come vincitore assoluto, registrando uno straordinario picco di **1293 GFLOPS** sulla configurazione 6000×2000 . Il massiccio utilizzo della *Shared Memory* azzerava i colli di bottiglia della VRAM. Degno di nota è anche il **Kernel WarpRow**, che, pur non utilizzando la memoria condivisa (sfruttando l'accumulo nei registri e il broadcast nativo), riesce a superare la barriera dei 1000 GFLOPS, dimostrandosi una strategia efficiente e con minor overhead di sincronizzazione.

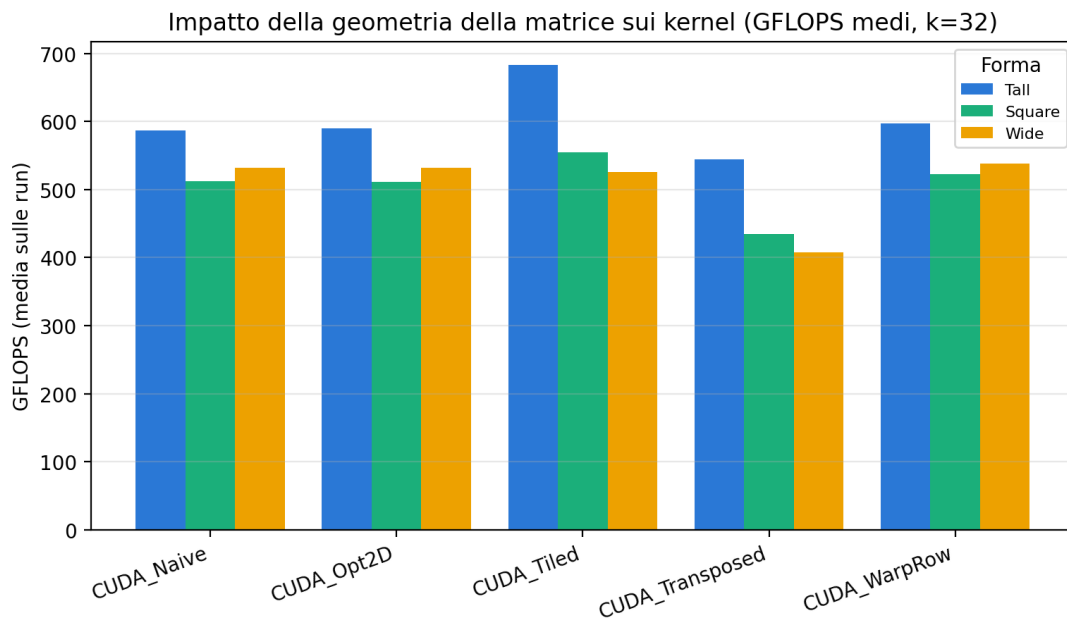


Figura 13: Impatto della geometria della matrice sul throughput a $k = 32$.

Analogamente alla CPU, anche l'architettura della GPU predilige matrici **Tall** (Figura 13). Una dimensione M molto ampia si traduce in un numero elevatissimo di blocchi e warp indipendenti, garantendo la saturazione ottimale dell'utilizzazione degli *Streaming Multiprocessors (SMs)*.

5.3 Comportamento Multi-Processo (MPI) e Oversubscription su GPU

Un aspetto cruciale del calcolo eterogeneo è il corretto interfacciamento tra il livello distribuito (MPI) e il dispositivo fisico.

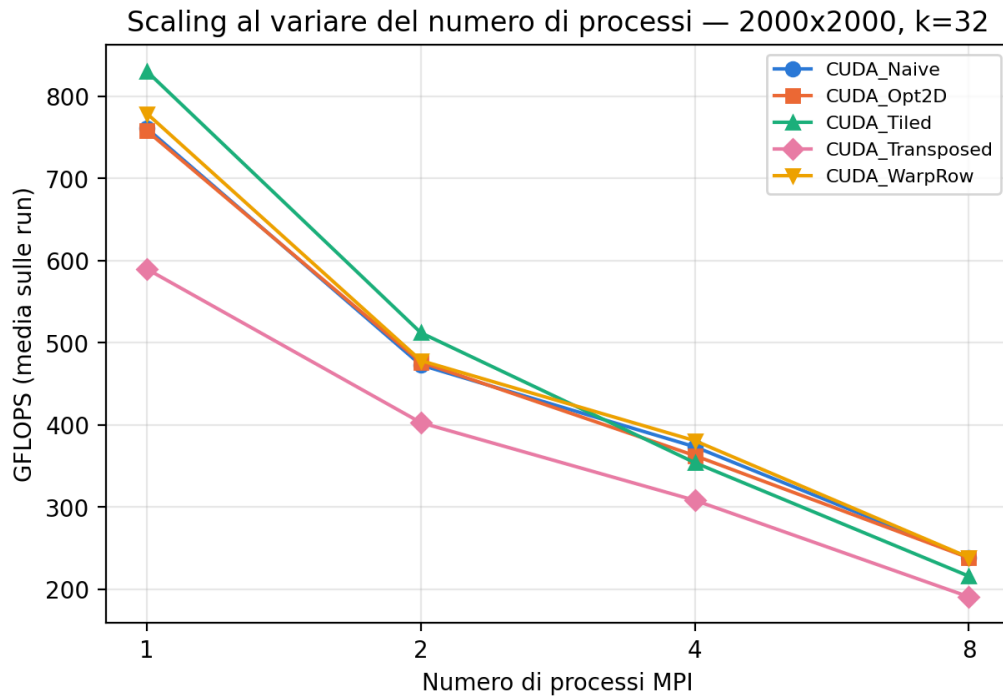


Figura 14: Degrado prestazionale (Negative Scaling) all'aumentare dei processi MPI su singola GPU condivisa.

La Figura 14 rappresenta uno stress-test architetturale (Oversubscription). All'aumentare dei processi logici MPI ($np \in \{1, 2, 4, 8\}$) mappati sull'unica GPU a disposizione, il throughput crolla drasticamente. Questo fenomeno è causato dalla competizione per il bus PCI-Express e dal context-switching. Processi distinti forzano lo scheduler hardware della GPU a continue *preemption*, serializzando i trasferimenti `cudaMemcpy`. Il fenomeno non è tuttavia universale: a valori di k elevati su matrici di grandi dimensioni si osserva un comportamento diverso. Ad esempio, per il kernel `Opt2D` sulla configurazione 4000×4000 , $k = 128$, il throughput a $NP = 4$ (1003 GFLOPS) supera quello a $NP = 1$ (953 GFLOPS), prima di ridiscendere a $NP = 8$ (891 GFLOPS). Una possibile spiegazione è che, a k grande, il working set locale per processo si riduce abbastanza da migliorare il riuso della shared memory, compensando parzialmente l'overhead di condivisione della GPU tra più contesti CUDA — un effetto che il caso a $k = 32$ di Figura 13, con working set locale già ridotto, non mostra. Il risultato pratico resta comunque chiaro: la strategia HPC più sicura, in assenza di conoscenza a priori del regime di k e delle dimensioni del problema, è assegnare un solo processo MPI per ogni acceleratore fisico disponibile; tuttavia, per problemi con k elevato, un moderato oversubscription può, in alcuni casi, risultare vantaggioso.

5.3.1 Impatto della Topologia di Griglia $P \times Q$

Oltre al numero di processi, la traccia del progetto richiede che la griglia $P \times Q$ sia configurabile esplicitamente dall'utente. Fissando $NP = 8$ e variando la sola forma della griglia (dalla configurazione "a striscia" 1×8 fino a 8×1 , passando per 2×4 e 4×2), a parità di $k = 32$, si osserva un degrado sistematico delle prestazioni al crescere di P (il numero di righe della griglia):

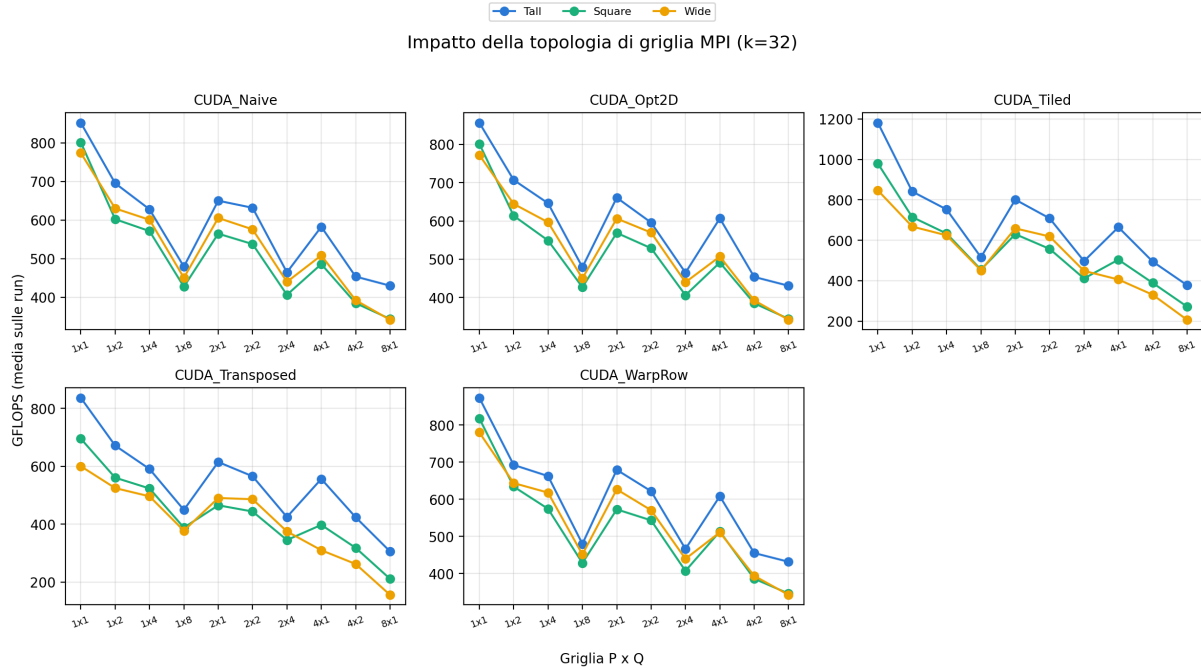


Figura 15: Impatto della forma della griglia MPI ($P \times Q$, a $NP = 8$ fissato) sul throughput dei kernel CUDA, $k = 32$.

Il kernel **Tiled** risulta il più sensibile in assoluto alla topologia: passando da 1×8 a 8×1 il throughput crolla da 474 a 286 GFLOPS, una perdita del 40%, contro un calo più contenuto per Naive, Opt2D e WarpRow. Questo comportamento è coerente con il funzionamento del kernel: aumentare P riduce M_{loc} (il numero di righe locali per processo), frammentando il prodotto matriciale locale in tile sempre più piccoli e limitando il riuso dei dati caricati in shared memory — l'ottimizzazione che rende Tiled il migliore in assoluto è anche, paradossalmente, la sua principale fonte di sensibilità alla frammentazione dei dati. Il risultato conferma quanto osservato per lo scaling puro del numero di processi: la scelta della griglia più efficiente non è indipendente dal kernel impiegato, ed è per questo che il progetto espone P e Q come parametri configurabili a runtime, anziché fissarli o lasciarli sempre al calcolo automatico di `MPI_Dims_create`.

5.4 Confronto Architetturale e Analisi End-to-End

Mantenendo il focus sul calcolo puro, il confronto tra i due paradigmi di calcolo evidenzia il netto dominio dell'acceleratore grafico per la classe di problemi *Dense Linear Algebra*.

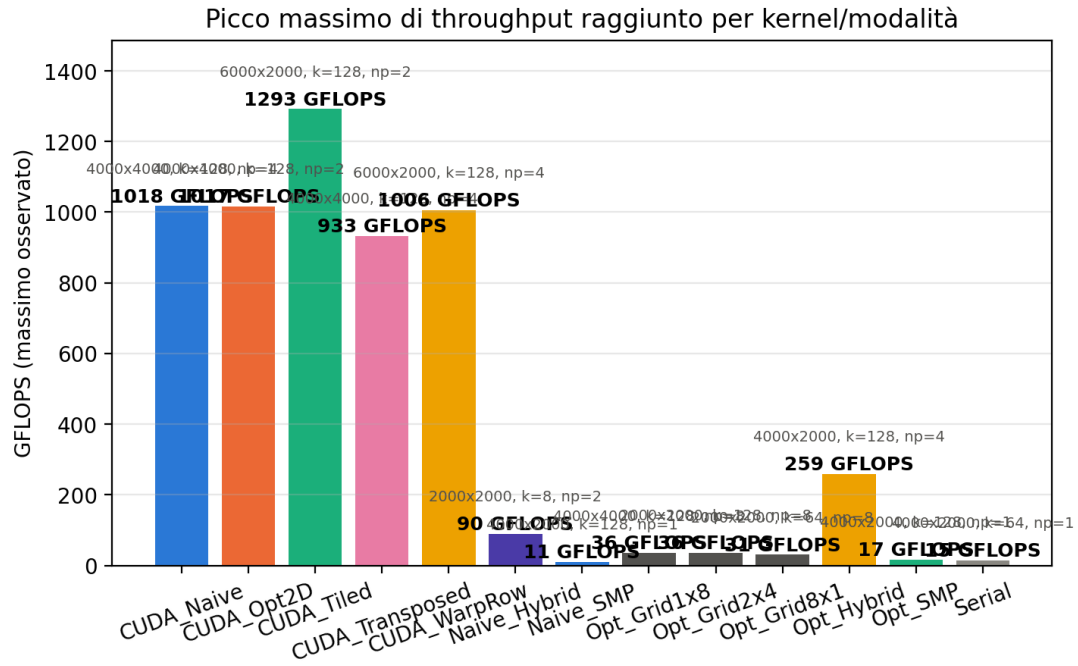


Figura 16: Confronto del picco massimo di throughput globale tra tutte le implementazioni CPU e GPU.

Come si nota in Figura 15, il divario di prestazioni (circa $5\times$ a favore della GPU rispetto alla migliore configurazione su CPU) riflette la differenza hardware tra le due architetture e la forza del modello SIMT sui calcoli vettoriali. Anche se i valori assoluti sono già stati analizzati, questo grafico aggregato rende visivamente chiare le proporzioni tra CPU e GPU.

Tuttavia, analizzando come si distribuisce il tempo di esecuzione totale (Time Breakdown), emerge un quadro diverso, fortemente limitato dalla velocità del bus di sistema.

Tempi di distribuzione, trasferimento e calcolo



Figura 17: Scomposizione dei tempi per singola run. In verde scuro i trasferimenti di memoria, in arancione il tempo di calcolo effettivo.

L'istogramma in Figura 17 (GPU nelle prime righe, CPU ibrida nell'ultima riga) sancì-

sce l'impatto critico del **Collo di Bottiglia PCI-Express**. Nell'approccio CUDA, la quasi totalità del tempo (barre verdi) è spesa per invocare le `cudaMemcpy` tra host e device, rendendo il tempo di calcolo puro pressoché irrilevante nell'economia dell'esecuzione complessiva. Al contrario, operando esclusivamente in RAM, la CPU spende la quasi totalità del suo tempo nell'elaborazione matematica utile (barre arancioni crescenti). Va inoltre notato che il tempo di distribuzione MPI del multivettore X risulta trascurabile rispetto al trasferimento H2D/D2H: in media 0.42 ms contro 119 ms, un rapporto di circa $280\times$. Il collo di bottiglia dell'applicazione risiede quindi nel bus PCI-Express, non nella rete di interconnessione MPI.

6 Conclusioni

Il progetto ha permesso di esplorare e validare sul campo le logiche del calcolo parallelo su architetture ibride eterogenee, confermando la totale correttezza algoritmica e la scalabilità della soluzione implementata.

Dall'analisi prestazionale e architetturale emergono tre considerazioni fondamentali:

- **Scalabilità Distribuita (MPI):** L'approccio *stateless* alla generazione dei dati e l'uso mirato di sotto-comunicatori (righe/colonne) hanno evitato la saturazione della RAM e della rete..
- **Gerarchia delle Memorie e Limite Memory-Bound:** Sia su CPU che su GPU, il calcolo $Y = A \cdot X$ con k contenuto si è dimostrato un problema fortemente limitato dalla banda di memoria. La vera sfida programmatica non è risieduta nella sola aritmetica, ma nell'orchestrazione degli accessi (vettorizzazione SIMD su host, *Shared Memory Tiling* e trasposizione della matrice sul device) per alimentare costantemente le componenti Hardware.
- **Il divario CPU-GPU e il limite PCI-Express:** Sebbene la GPU abbia dominato in termini di calcolo puro (raggiungendo i 1293 GFLOPS contro i 259 GFLOPS della migliore configurazione ibrida NUMA su host), l'analisi dei tempi *End-to-End* ha fortemente ridimensionato questo risultato. I trasferimenti *Host-to-Device* sul bus PCI-Express si sono rivelati il vero collo di bottiglia del sistema, monopolizzando i tempi di esecuzione.

In sintesi, l'impiego di un acceleratore hardware isolato per questa specifica classe di problemi produce un vantaggio operativo reale solo a due condizioni: un'intensità aritmetica sufficientemente alta (valori di k molto elevati in grado di mascherare le latenze), oppure l'inserimento del calcolo all'interno di una pipeline HPC più ampia, in cui i dati risiedano in modo persistente nella memoria senza necessitare di continui trasferimenti sul bus di sistema.